Get unlimited access to the best of Medium for less than $1/week. Become a member

V8 optimizations behind the scenes



Yash Suresh Chandra Follow 8 min read · Jul 23, 2019

219

This article aims to scratch the surface of how V8 (or any modern JS engine nowadays) uses the power of **inline caching** and **hidden classes** to optimize execution of JavaScript code under the hood.

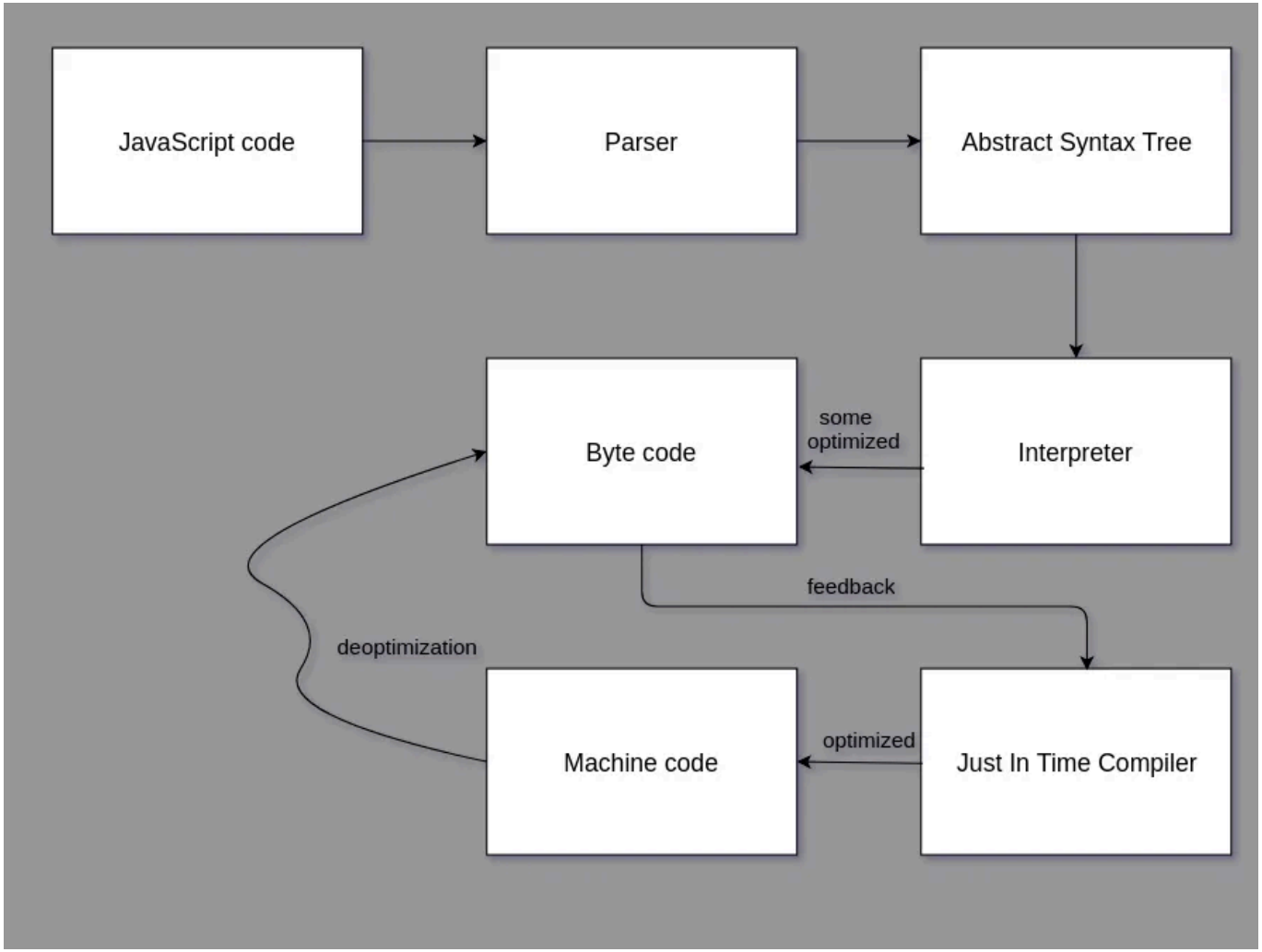
A little about JavaScript Engines

JavaScript engines are computer programs that takes your JS code and executes it with the help of runtime environment (eg. browsers, node). JS engine mainly consists of -

1. *Parser* — takes source code and converts it into Abstract Syntax Tree
2. *Compiler and interpreter* — to execute parsed JS code.
3. *Call stack* — a data structure to store state of program.
4. *Heap* — memory allocation area.
5. *Garbage collector* — to free up unused memory.

Some of the popular JS engines are *V8* in Google Chrome and Node, *Spidermonkey* in Mozilla Firefox, *Chakra* in Microsoft Edge.

Almost all JS engines nowadays perform similar steps. They take JS code and parse it to construct an *Abstract Syntax Tree*. This AST is fed into an interpreter that produces slightly optimized bytecode (using function inlining, etc). This makes your application start up really quick. However, there is more scope of optimizations that JS engine can perform (we shall see soon). To do this, there is an additional *Just In Time* (JIT) compiler which continously takes feedback from the code running and generate highly optimized machine code based on some assumptions. If at any point of time, assumption turns out to be incorrect, there is also a deoptimization step.



JavaScript code compilation

A little about V8

V8 is Google's open source high-performance JavaScript and WebAssembly engine, written in C++. It is used in Chrome and in Node.js, among others.

V8 has an interpreter *Ignition* that produces some optimised bytecode for fast application start up and a Just In Time compiler *TurboFan* that keeps running in background and generates optimized machine code for faster execution. V8 uses many different techniques to optimize machine code. In this post we will see one of them.

But first, some background.

Memory allocation in V8

Javascript is a dynamically typed language. Type of a variable is known only at runtime and type of a variable can change during it's life cycle. This also opens up the possibility of adding or deleting properties from an object. These arbitrary events make it really hard for JS engines to determine the memory address of the value of a property inside an object.

> *In static typed languages like Java, compiler knows at compile time what will be the layout of the object. Since object structure cannot change after being defined, compiler can store the offsets of the location of property in memory. So any object's property value can be retrieved in one attempt (by checking in memory at stored offset). This cannot happen in dynamically typed languages because object structure can change at runtime.*

One technique would be to use a **hash** based dictionary to store memory address of value of a property with key as property name.

It would look something like -

```
// get value of a property
function getKeyValue(key) {
   let address = calculateAddressOfKey(key);
   // get value from memory address
}
// set value of a property
function setKeyValue(propName, value) {
   let address = calculateAddressOfKey(key);
   // set value for key at memory address
}
// calculate hash of key
function calculateAddressOfKey(key) {
   let memoryAddress = getHash(key); // calculate hash
   return memoryAddress;
}
```

The above code works, but one major issue is every time we call *getKeyValue* or *setKeyValue*, we in turn use *getHash* function, which can be computationally expensive.

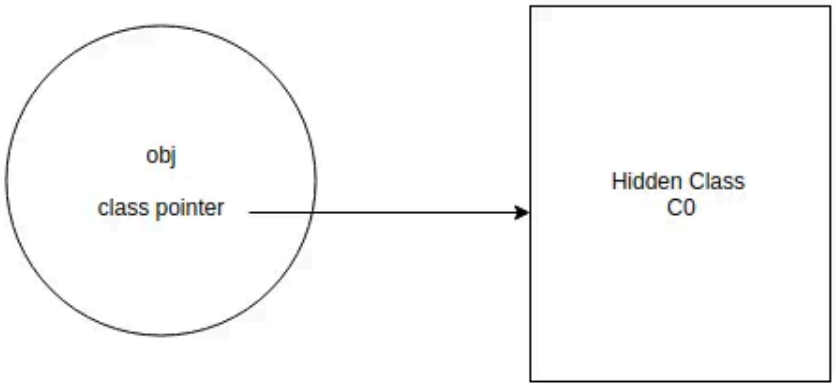
V8 does not use the above method. Instead it takes care of object structure with the help of **hidden classes**.

Hidden classes

Hidden classes store the offsets of all the properties of an object like a fixed layout similar to Java except that hidden classes are created runtime. Every object corresponds to a hidden class (internally called *shape/map*). This works in the following ways -

1. When an object is created, an empty hidden class is attached to it.

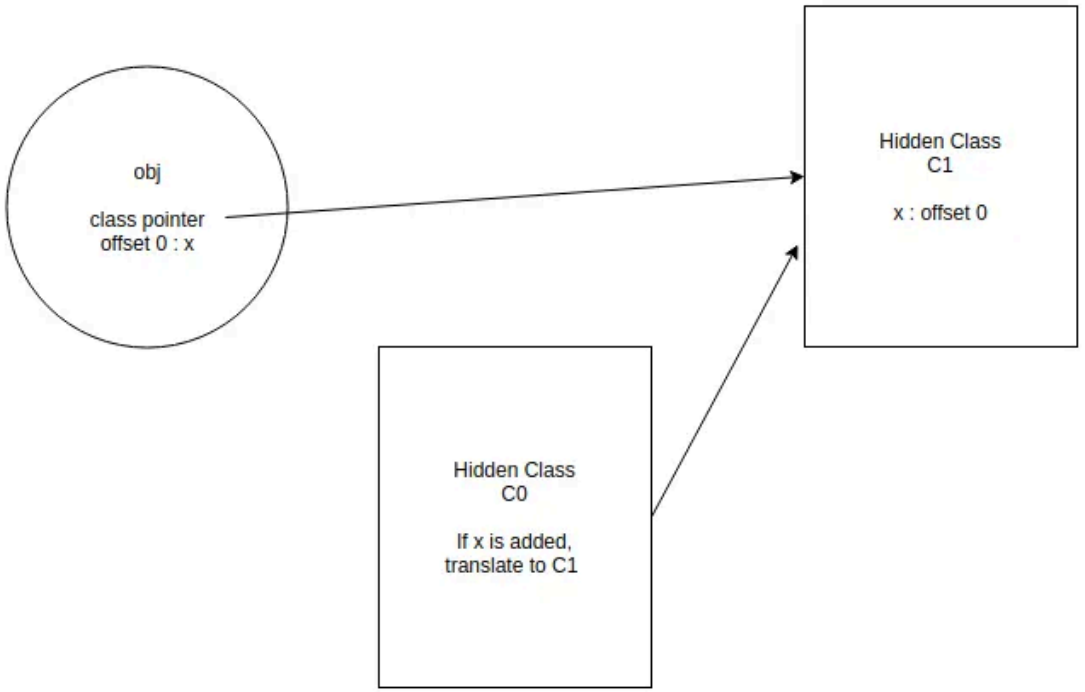
```
var obj = {};
```



obj = {}

2. Every time a new property is added to the object, a transition takes place where now object starts pointing to a new hidden class which is referred by old hidden class.

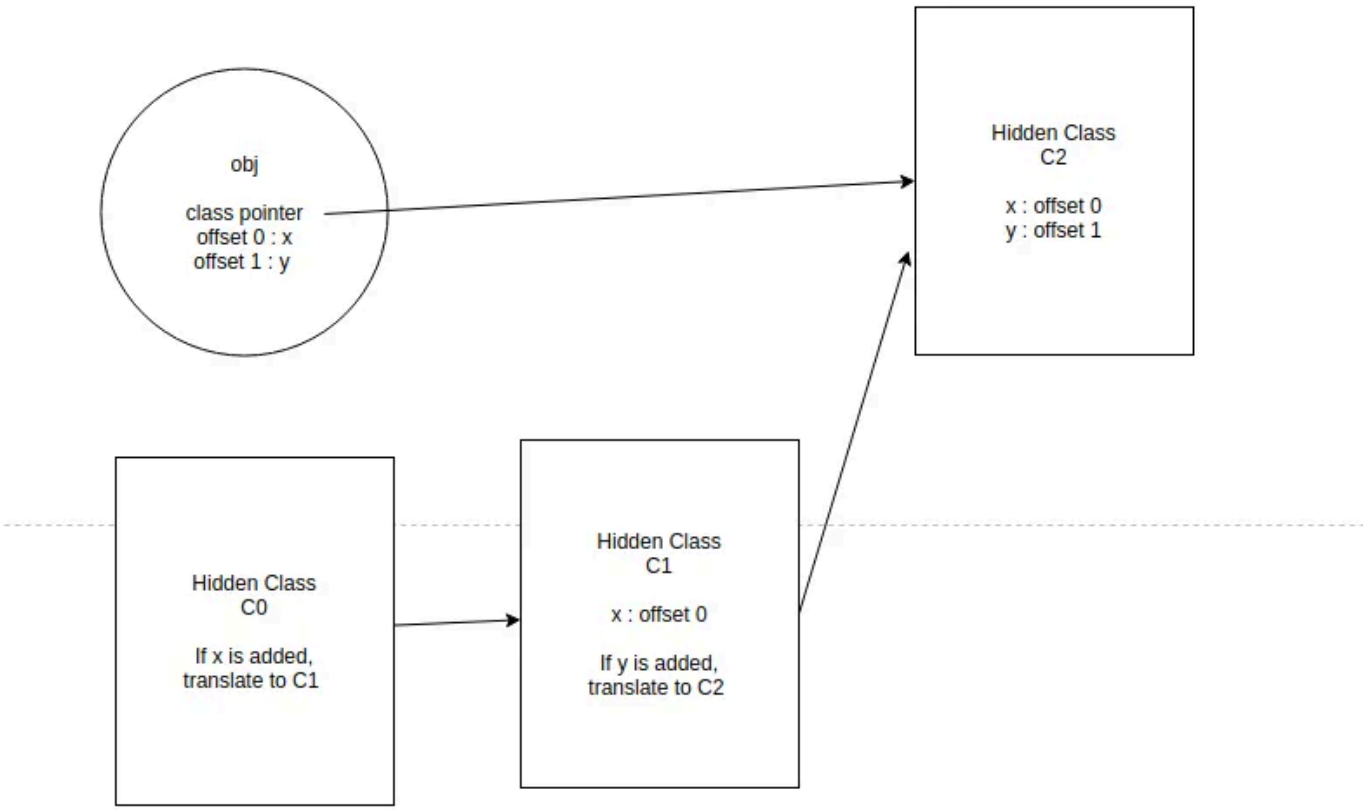
```
obj.x = 1;
```



obj = {x : 1}

3. Each hidden class stores the offset to the newly added property in the object which now points to a new hidden class.

```
obj.y = 2;
```



obj = {x : 1, y : 2}

This makes retrieval of memory address of a property much easier and less expensive. JS engine just require to find the appropiate hidden class, then it can get all the required offsets.

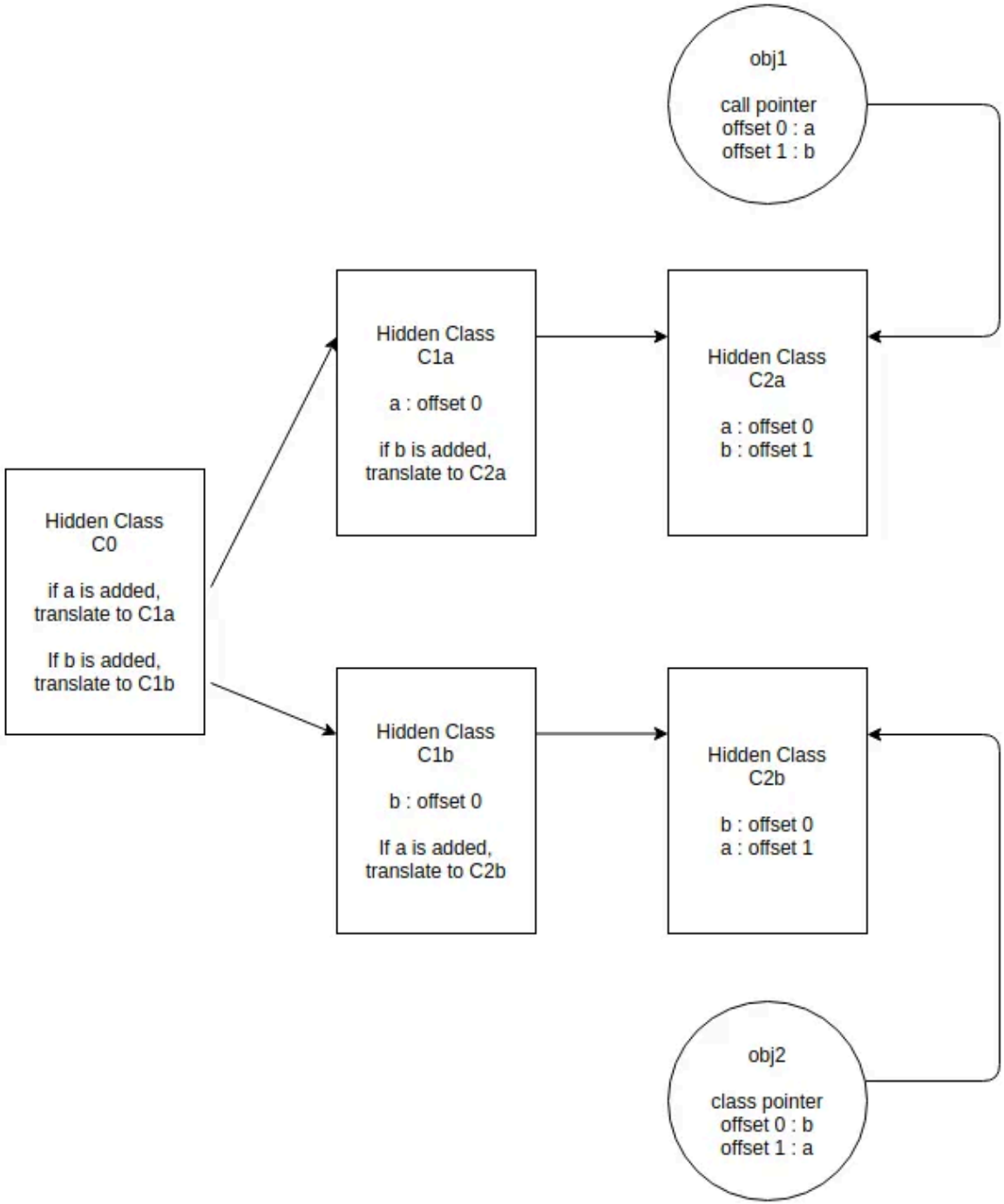
Since each transition in properties is recorded, hidden class transitions form a tree like structure with all objects pointing to any one of the node.



One thing to note here is that these hidden classes can be **shared** among objects. So same objects points to same hidden class. However, for two objects to be same, assignment order of their properties must also be same.

```
var obj1 = {};
var obj2 = {};
obj1.a = 1;
obj1.b = 2;
obj2.b = 2;
obj2.a = 1;
```

In the above code example, *obj1* and *obj2* will have different hidden classes even though they are structurally same because there corresponding hidden classes took different transition paths. For above code block, hidden classes would be stored like -


obj1
call pointer
offset 0 : a
offset 1 : b
Hidden Class
C1a
a : offset 0
if b is added,
translate to C2a
Hidden Class
C2a
a : offset 0
b : offset 1
Hidden Class
C0
if a is added,
translate to C1a
If b is added,
translate to C1b
Hidden Class
C1b
b : offset 0
If a is added,
translate to C2b
Hidden Class
C2b
b : offset 0
a : offset 1
obj2
class pointer
offset 0 : b
offset 1 : a

Here, order of assignments of *a* and *b* are different for *obj1* and *obj2*, so there hidden classes are different.

> *V8 does not always start with an empty hidden class. Engines apply some optimizations for object literals that already contain properties. For obj = {a :1}, it starts with a hidden class that contains offset of 'a'.*

Inline Caching

Inline caching is an optimization technique that relies upon the observation that repeated calls to same function tends to occur on same type of objects. This is a huge help in dynamically typed languages like JS where objects can change at runtime because now compiler can assume the type of object at the call site, store that type in cache. The next time when same function is called, compiler can directly search for the type in cache.

```
var x = {a : 1, b : 2};

/*
at some point compiler will realize obj structure is always like:
{a : <int>, b : <int>}
```

```
*/
function process(obj) {

   //process obj
}
for(let i=0; i<1000; i++) {

   process(x);
}
```

There are broadly 3 types of inline caching —

1. *Monomorphic* (optimized) — always same type of objects are passed.
2. *Polymorphic* (slightly optimized)—limited number of different types of objects passed.
3. *Megamorphic* (unoptimized) — any number of different objects passed.

Optimization

V8 uses both *hidden classes* and *inline caching* to optimize **hot** functions. Hot functions here are those functions which are called repeatedly with same type of objects (objects with same hidden class). When a function becomes hot, instead of searching for object's hidden class every time function is called, V8 stores the offsets of different properties of the object so subsequent calls does not check for hidden class but directly access

properties using stored offsets. This increases execution speed because there is no need to look for object's hidden class.

V8 **optimistically** optimizes code execution. It can show significant improvements.

However, as the saying goes...



quote

Responsibility

We discussed how V8 hopes to find same type of object as argument for a hot function. But since JS is a dynamically typed language, the argument object's shape may change. When this happens, it triggers deoptimization of machine code because now V8 cannot directly lookup for stored offsets to find properties. This causes problems as now if the function is called, appropiate hidden class should be searched. Now, again if the function is called with some parameters, V8 cannot directly assume its hidden class and hence its layout, instead it looks into the memory and find the matching hidden class for this object to access its properties later.

Experiment

Above optimization and deoptimization concepts can be demonstrated using fairly simple code block.

```
function processGeneric(n, point) {
    let sum = 0;
    for(let i=0; i<n; i++) {
        sum += point.a;
    }
    return sum;
}

function processPoint2(n, point) {
    let sum = 0;
    for(let i=0; i<n; i++) {
        sum += point.a;
    }
    return sum;
}

function processPoint(n, point) {
    let sum = 0;
    for(let i=0; i<n; i++) {
        sum += point.a;
    }
    return sum;
}

function runMain() {
    let warmup = 500;
    let N = 120000000;
    let p1 = {};
    let p2 = {};
    let p3 = {};
    p1.a = 1;
    p2.b = 2;
    p2.a = 1;
```

```
    p3.a = 1;
    let sum;
    let start, t1, t2, t3;
    sum = processPoint(warmup, p3);
    start = new Date();
    sum = processPoint(N, p1);
    t1 = new Date() - start;
    sum = processPoint2(warmup, p2);
    start = new Date();
    sum = processPoint2(N, p1);
    t2 = new Date() - start;
    sum = processGeneric(warmup, 1);
    start = new Date();
    sum = processGeneric(N, p1);
    t3 = new Date() - start;
    console.log(t1, t2, t3);
}

(function() {
    runMain();
})();
```

v8_hidden_class_inline_caching_benchmarks.js hosted with ❤ by GitHub view raw

Before looking into results, just try to guess the output of the above code (hint : no trick question)

Results

On my machine upon running it 10 time, outputs are -

```
122 196 237
122 198 238
121 198 237
120 197 237
121 197 238
120 196 252
120 198 236
120 198 237
120 197 238
119 197 238
```

This was expected because passing same type of object helps V8 to optimize whereas passing different types of objects causes deoptimizations.

In the above code, *processPoint* is always called with objects (p1 and p3) having same hidden class, *processPoint2* is called with objects (p2 and p1) having different hidden classes. *processPoint3* is called with generic variables (p1 object and integer 1) and is even slower than *processPoint2*.

Takeaways

We noticed that V8 (and other JS engines) have implemented pretty good techniques to improve performance of this dynamically typed language. But it also depends upon us, to use this **power** by writing code **responsibly**.

Some steps we can follow to bring this power into good use are -

1. Initialize object properties in same order so that a hidden class can be shared — *less hidden classes*.
2. Always call functions with same type of objects so that inline caching can be used for optimization — *functions remains hot*.
3. After initializing an object, don't add properties so that no hidden class transition takes place — *faster hidden class search*.

Closing

There are lots of topics that we did not discuss here. For example, maintaining states of inline caches for each function, On stack replacement (OSR) of hot functions, hidden classes for arrays, tagging/untagging of elements in arrays. There exists a whole bunch of optimizations out there that we can use to write better and faster code. We can aim to learn as much of these techniques to get maximum out of what advantages JS engines have to offer.

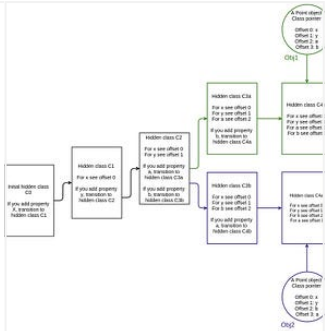
Happy coding!

References

Javascript Hidden Classes and Inline Caching in V8

Javascript is a dynamic programming language which means that properties can easily be added or removed from an object...

richardartoul.github.io



JavaScript engine fundamentals: Shapes and Inline Caches

This article describes some key fundamentals that are common to all JavaScript engines - and not just V8, the engine...

mathiasbynens.be

How JavaScript works: inside the V8 engine + 5 tips on how to write optimized code

Couple of weeks ago we started a series aimed at digging deeper into JavaScript and how it actually works: we thought...

blog.sessionstack.com



JavaScript Engines Hidden Classes (and Why You Should Keep Them in Mind)

A previous version of this post was published by accident 10 days ago. I didn't correct this mistake because it had...

draft.li

JavaScript V8 Optimization Js Internals



Written by Yash Suresh Chandra

18 followers · 0 following

Fat and unimportant

Follow

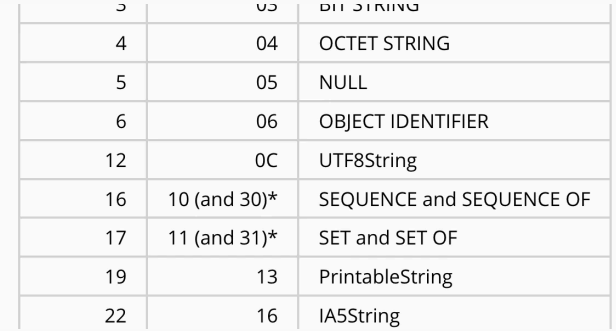
No responses yet



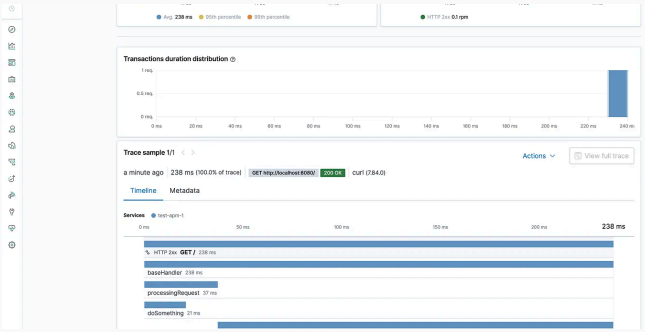
Arunkumargole

What are your thoughts?

More from Yash Suresh Chandra



3	03	BIT STRING
4	04	OCTET STRING
5	05	NULL
6	06	OBJECT IDENTIFIER
12	0C	UTF8String
16	10 (and 30)*	SEQUENCE and SEQUENCE OF
17	11 (and 31)*	SET and SET OF
19	13	PrintableString
22	16	IA5String



Yash Suresh Chandra

Anatomy of a PEM file

Let's see how a PEM file is constructed and what does its content mean. We will use an...

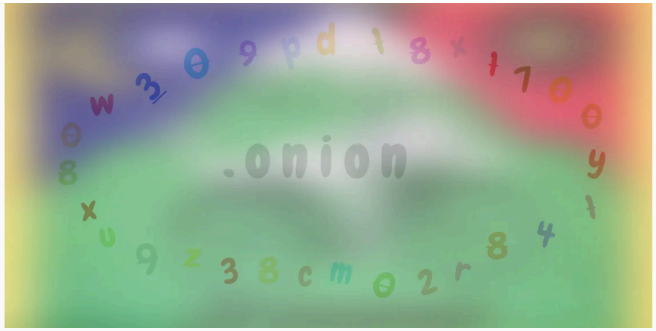
Aug 26, 2023 29

Yash Suresh Chandra

Integrate Elastic APM in Golang servers

Introduction

Mar 6, 2023 3

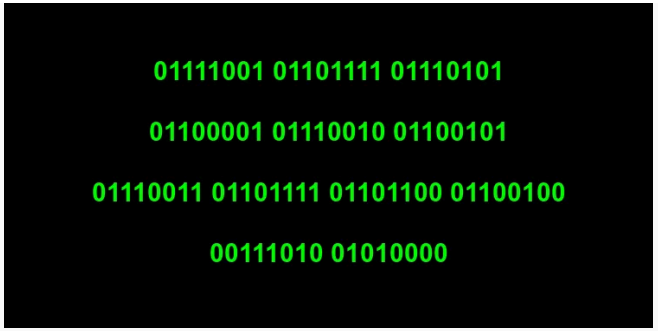


Yash Suresh Chandra

Practical Things: How to generate pseudorandom onion addresses

Disclaimer

Jun 14, 2020 50



Yash Suresh Chandra

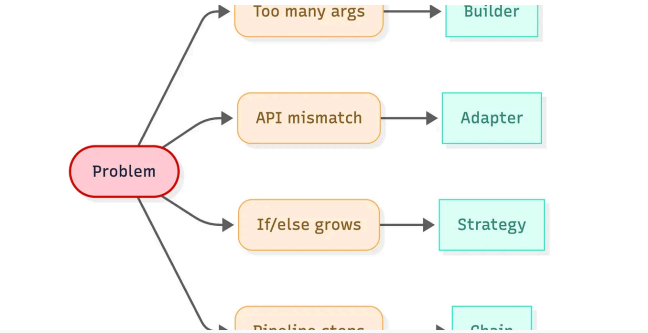
An inner-view to Protobuf encoding

In this article we will see how protocol buffers encodes data to make it more compact...

Mar 21, 2020 96 3

See all from Yash Suresh Chandra

Recommended from Medium

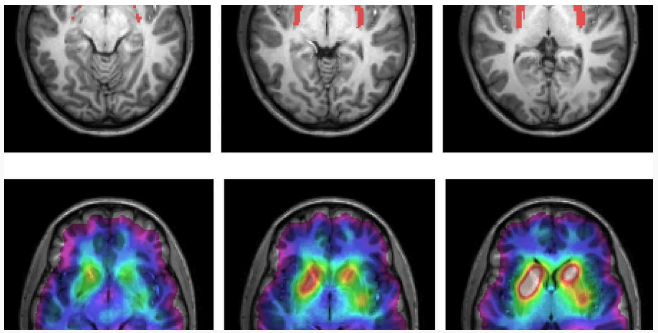


In Women in Technology by Alina Kovtun ✨

Stop Memorizing Design Patterns: Use This Decision Tree Instead

Choose design patterns based on pain points: apply the right pattern with minimal over-...

✨ Jan 29 👏 3.4K 💬 31

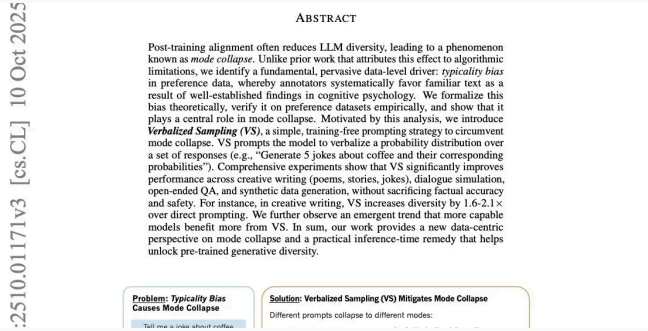


In Write A Catalyst by Dr. Patricia Schmidt

As a Neuroscientist, I Quit These 5 Morning Habits That Destroy You...

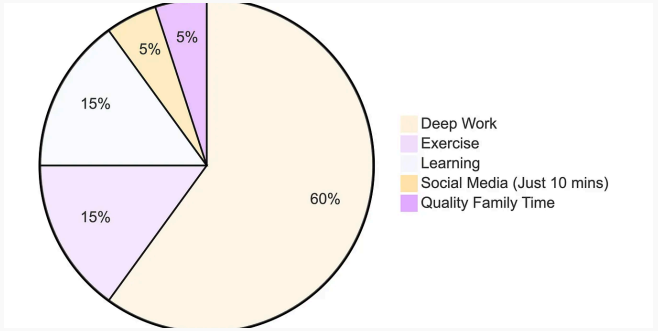
Most people do #1 within 10 minutes of waking (and it sabotages your entire day)

✨ Jan 15 👏 30K 💬 522



In Generative AI by Adham Khaled

Stanford Just Killed Prompt Engineering With 8 Words (And I...



In Level Up Coding by Teja Kusireddy

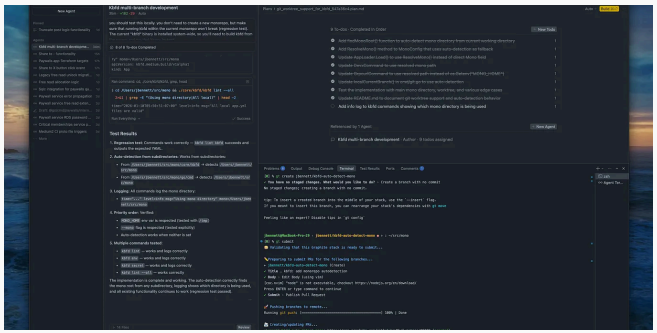
I Stopped Using ChatGPT for 30 Days. What Happened to My Brai...

ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...

Oct 20, 2025 24K 631

91% of you will abandon 2026 resolutions by January 10th. Here's how to be in the 9% who...

Dec 28, 2025 6.1K 250

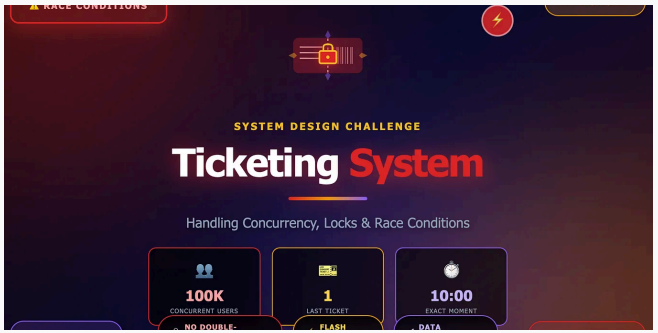


Jacob Bennett

The 5 paid subscriptions I actually use in 2026 as a Staff Software...

Tools I use that are (usually) cheaper than Netflix

Jan 19 2.9K 75



Arvind Kumar

Building a Ticketing System: Concurrency, Locks, and Race...

What happens when 100,000 fans try to book the same concert ticket at exactly 10:00 AM...

Oct 30, 2025 1.7K 18

See more recommendations